Seguridad de Archivos en la Capa del Kernel y Mapeo de Memoria (mmap)

Juan Pablo Lopera Arrazola · Samuel Rendón Pabón
Sistemas Operativos SO2026B (EAFIT)
Noviembre de 2026

Contenido

Informe — Seguridad de Archivos mediante Intervención en la Capa del Kernel y Mapeo de Memoria (`mmap`)
PARTE I — ANÁLISIS CONCEPTUAL (10 PREGUNTAS DIAGNÓSTICAS)

- 1. Mapeo virtual y *page faults*: secuencia exacta desde `mmap()` hasta el primer fallo
- 2. `MAP_SHARED` vs. `MAP_PRIVATE` en `vm_area_struct` e implicaciones de seguridad
- 3. Ciclo de vida de una página bajo COW y riesgo por mala gestión de banderas
- 4. Intercepción en el kernel: LKM con *hook* en la *syscall table* vs. LSM (`file_mmap`)
- 5. Cifrado transparente en memoria: interacción Page Cache ↔ driver de FS
- 6. Sincronización e integridad (`msync` / `fsync`) y evasión de filtros
- 7. Control de accesos con eBPF (`kprobes` / `fexit`) sin recompilar el núcleo
- 8. Evasión de permisos POSIX: `O_RDONLY` mapeado con `PROT_READ | PROT_WRITE | MAP_PRIVATE`
- 9. Mitigación de carreras entre `madvise(MADV_DONTNEED)` y escrituras sobre páginas privadas
- 10. Aislamiento de memoria: `struct page` , tablas de páginas y bits de la MMU

PARTE II — DISEÑO DE LA ESTRATEGIA CON APOYO DE INTELIGENCIA ARTIFICIAL

- 2.1 Flujo de trabajo asistido por IA
- 2.2 Ingeniería de *prompts* y verificación crítica
- 2.3 Alucinaciones técnicas detectadas y corregidas
- 2.4 Procedimiento de validación manual
- 2.5 Arquitectura seleccionada
- 2.6 Matriz de compensaciones (*trade-offs*)

PARTE III — IMPLEMENTACIÓN, RESULTADOS Y ARGUMENTACIÓN

- 3.1 Entregables de código
- 3.2 Justificación teórica: ¿por qué en la capa de kernel?
- 3.3 Medición de impacto (metodología y salida de referencia)
- 3.4 Limitaciones y trabajo futuro
- 3.5 Argumentación final
- Anexo — Rúbrica y cómo este entregable la cubre

Informe — Seguridad de Archivos mediante Intervención en la Capa del Kernel y Mapeo de Memoria (`mmap`)

> **Asignatura:** Sistemas Operativos — SO2026B (EAFIT) **Integrantes:** Juan Pablo Lopera Arrazola (GitHub `Juanp-la`) · Samuel Rendón Pabón **Solución implementada:** `MMAPGuard` (módulo de kernel) — ver `README.md` y carpeta `kernel/`. **Repositorio:** github.com/Juanp-la/mmapguard-so2026b **Referencia base:** SO2026B/USOS/CapaKernel

---

PARTE I — ANÁLISIS CONCEPTUAL (10 PREGUNTAS DIAGNÓSTICAS)

1. Mapeo virtual y *page faults*: secuencia exacta desde `mmap()` hasta el primer fallo

`mmap()` **no asigna memoria física**; solo describe una región. La secuencia es:

Fase A — Establecimiento (syscall `mmap`) 1. `sys_mmap` → `ksys_mmap_pgoff` → `vm_mmap_pgoff` → `do_mmap`. 2. `get_unmapped_area()` elige el rango virtual libre; `mmap_region()` reserva e inicializa una `struct vm_area_struct` **(VMA)**: fija `vm_start`, `vm_end`, `vm_pgoff`, `vm_file` y traduce `PROT_*` / `MAP_*` a `vm_flags` (`VM_READ`, `VM_WRITE`, `VM_SHARED`, `VM_MAYWRITE`, ...). 3. `call_mmap()` invoca `file->f_op->mmap()` — en nuestra solución, `dev_mmap()`, que valida la política y asigna `vma->vm_ops`. **No se puebla ninguna PTE.** 4. El VMA se inserta en el árbol de VMAs del proceso (`mm->mm_mt`, *maple tree* en 6.x) y la syscall retorna el puntero virtual.

Fase B — Primer acceso (fallo de página por hardware) 5. El proceso lee/escribe `ptr[i]`. La **MMU** recorre las tablas `PGD→P4D→PUD→PMD→PTE`; la PTE está vacía → la CPU dispara la excepción `#PF` **(vector 14)** y deja la dirección en `CR2`. 6. `exc_page_fault()` → `do_user_addr_fault()` → `find_vma()` localiza el VMA y verifica que el acceso sea compatible con `vm_flags` (si no, `SIGSEGV`). 7. `handle_mm_fault()` → `__handle_mm_fault()` → `handle_pte_fault()`. Como el VMA tiene `vm_ops->fault`, entra por `do_fault()` → `__do_fault()` → **`vma->vm_ops->fault()`** (nuestro `mg_vma_fault`), que devuelve una `struct page`. 8. El kernel instala la PTE con `set_pte_at()`, actualiza la TLB y **reanuda la instrucción**. Desde aquí, los accesos siguientes a esa página ocurren a velocidad de hardware, sin volver a Ring 0.

2. `MAP_SHARED` vs. `MAP_PRIVATE` en `vm_area_struct` e implicaciones de seguridad

La diferencia se codifica en `vma->vm_flags`:

- `MAP_SHARED` activa `VM_SHARED`. Las escrituras se reflejan sobre las páginas de la **Page Cache** del *inode* (`vma->vm_file->f_mapping`), son **visibles para todos** los procesos que mapeen el mismo archivo y se **persisten** a disco por *writeback*. *Implicación:* canal de visibilidad cruzada y persistencia; un proceso con acceso de escritura altera lo que ven los demás y el archivo real.
- `MAP_PRIVATE` no activa `VM_SHARED` pero sí `VM_MAYWRITE`. Usa **Copy-On-Write**: la primera escritura crea una **copia privada** anónima; el cambio **no se persiste** ni es visible para otros. *Implicación:* el riesgo no es persistencia sino **fuga de lectura** (puedo leer todo el contenido) y **alteración temporal** de mi vista local, que ha sido pieza de cadenas de exploits (Dirty COW).

En `MMAPGuard` esto importa porque el modo `readonly` retira `VM_WRITE` y `VM_MAYWRITE` del VMA (`vm_flags_clear`), cerrando tanto la escritura compartida como la copia privada escribible.

3. Ciclo de vida de una página bajo COW y riesgo por mala gestión de banderas

Ciclo: tras `fork()`, padre e hijo comparten las páginas marcando la **PTE como solo lectura** y con `_mapcount > 1`; la página se marca COW. Cuando cualquiera escribe → `#PF` → `do_wp_page()`: si la página es exclusiva se reutiliza; si es compartida se **copia** (`copy_page`), se instala una PTE nueva **escribible** y se decrementa el `_mapcount` de la original.

Riesgo: si el kernel decide la escritura con base en banderas mal gestionadas (`VM_MAYWRITE`, bit *dirty* de la PTE, `FOLL_FORCE`), un atacante puede lograr que una escritura destinada a su **copia privada COW** termine aplicándose sobre la **página original de la Page Cache** de un archivo de solo lectura propiedad de `root` (p. ej. `/etc/passwd`). Eso corrompe un archivo del sistema y deriva en **escalada de privilegios** — exactamente la clase de **Dirty COW (CVE-2016-5195)**. La defensa correcta no confía en el bit *dirty* para decidir reutilización y rastrea la exclusividad real de la página (`PageAnonExclusive`, kernels ≥ 5.19).

4. Intercepción en el kernel: LKM con *hook* en la *syscall table* vs. LSM (`file_mmap`)

Criterio	LKM parcheando `sys_call_table`	LSM (`security_mmap_file`)
Mecanismo	Localizar la tabla (`kallsyms`), **deshabilitar el bit WP de** `CR0` y reescribir punteros.	API oficial de ganchos; se registra con `security_add_hooks()` .
Estabilidad	Frágil: KASLR, tabla de solo lectura, *CR0 pinning*/CET lo rompen.	Estable y soportada entre versiones.
Seguridad	Peligrosa: carrera al escribir la tabla, puede causar *panic*; el propio *hook* es un vector.	Diseñada para seguridad; apilable con SELinux/AppArmor.
Granularidad	Intercepta la syscall completa, sin contexto rico.	Recibe `struct file` , `prot` , `flags` ; puede retornar `-EACCES` .
Viabilidad	Baja; didáctica pero no recomendable en producción.	**Alta — es el camino recomendado.**

Conclusión: para archivos arbitrarios, un **LSM** (o **BPF-LSM**) es superior. `MMAPGuard` usa el `.mmap` **del propio driver**, que es el punto de intercepción natural y limpio para un dispositivo, evitando por completo el parcheo de la tabla de syscalls.

5. Cifrado transparente en memoria: interacción Page Cache ↔ driver de FS

El patrón de referencia es **fscrypt**: el archivo está cifrado en disco; cuando un proceso **autorizado** lo lee, la ruta de lectura del sistema de archivos (`read_folio` / `readahead`) **descifra hacia las páginas de la Page Cache** asociadas al `address_space` del *inode*, usando una clave ligada al **keyring** y a las credenciales. `mmap()` proyecta esas páginas ya en claro. Para **no exponer** el texto descifrado a procesos no privilegiados:

- La clave vive en el *keyring* del usuario autorizado, no en las páginas.
- El control de acceso del *inode* (DAC/LSM) decide quién puede abrir/mapear.
- Idealmente, claves **por archivo** y borrado de claves al revocar acceso.

`MMAPGuard` reproduce la idea a escala didáctica: el contenido vive **cifrado** en el almacén de respaldo y el método `.fault` **descifra página a página solo para el** `uid` **autorizado**; un proceso no autorizado recibe `SIGBUS` y jamás ve el claro.

6. Sincronización e integridad (`msync` / `fsync`) y evasión de filtros

Una página modificada en memoria se marca **sucia**: el hardware activa el **bit *dirty* de la PTE** y el kernel marca `PG_dirty` en la `struct page` . La persistencia ocurre por *writeback* (hilos *flusher*), o se fuerza con `msync(MS_SYNC)` (región mapeada) y `fsync()` (descriptor).

Evasión: si un filtro de seguridad solo inspecciona en la syscall `write()` , un atacante que usa `mmap()` **nunca pasa por ese punto**: modifica el contenido con instrucciones `MOV` directas (sin syscalls) y lo sincroniza después, o lo deja solo en RAM. Por eso la inspección debe engancharse en `page_mkwrite` **/ writeback /** `.fault` , no en `write()` . `MMAPGuard` decide la política de escritura en `.mmap` (en modo solo lectura retira `VM_WRITE` / `VM_MAYWRITE` , de forma que ninguna escritura llega a generar un fallo) y recifra el contenido al cerrar el VMA (`.close`). Se evita a propósito un manejador `page_mkwrite` : ese *hook* exige que la página pertenezca a un `address_space` (`page->mapping != NULL`), y las páginas propias del driver (de `alloc_page()`) no lo cumplen; devolver desde `page_mkwrite` haría que el kernel las tratara como *truncadas* y reintentara el fallo en bucle. Es una restricción real del contrato del kernel, documentada aquí como decisión de diseño.

7. Control de accesos con eBPF (`kprobes` / `fexit`) sin recompilar el núcleo

eBPF permite instrumentar **sin recompilar ni reiniciar**:

- **Observación/auditoría:** `kprobe` / `fentry` sobre `handle_mm_fault` , `security_mmap_file` o `vma_set_page_prot` , y `fexit` / `kretprobe` para ver el valor de retorno. Patrones por PID se acumulan en **mapas BPF** y se exportan a usuario por **ring buffer**.
- **Control (allow/deny):** programas `BPF_PROG_TYPE_LSM` adjuntos a `file_mmap` pueden **retornar** `-EACCES` y bloquear el mapeo.

Límites: eBPF es excelente para **detectar y decidir**, pero el verificador restringe lo que puede hacer; **no está pensado para transformar el contenido** (descifrar/cifrar páginas). Para eso sigue siendo necesario un módulo o la capa de FS. Combinación ideal: **BPF-LSM para política + módulo para transformación**.

8. Evasión de permisos POSIX: `O_RDONLY` mapeado con `PROT_READ | PROT_WRITE | MAP_PRIVATE`

Sí se permite. El kernel exige `FMODE_WRITE` únicamente para `MAP_SHARED` **+** `PROT_WRITE` (ahí la escritura llegaría al archivo). Para `MAP_PRIVATE` lo autoriza aunque el descriptor sea `O_RDONLY` , porque las escrituras caen sobre **páginas COW anónimas privadas** que **nunca tocan el archivo**.

Riesgos: (a) **fuga de información** — puedo leer íntegro un archivo que solo tenía permiso de lectura y manipular su contenido en mi espacio; (b) **alteración temporal** que puede confundir validaciones basadas en la vista en memoria; (c)

históricamente, es el eslabón inicial de la cadena **Dirty COW**. `MMAPGuard` lo neutraliza: en modo `readonly` retira `VM_MAYWRITE`, de modo que ni el mapeo escribible ni un `mprotect(PROT_WRITE)` posterior prosperan (`test_readonly.c`).

9. Mitigación de carreras entre `madvise(MADV_DONTNEED)` y escrituras sobre páginas privadas

`madvise(MADV_DONTNEED)` **vacía las PTEs** del rango (`zap_page_range`), forzando que el siguiente acceso vuelva a fallar. El exploit clásico de Dirty COW corría un hilo haciendo `MADV_DONTNEED` mientras otro escribía vía `/proc/self/mem` / `FOLL_FORCE`, buscando una ventana donde la escritura cayera en la página compartida. El kernel actual lo previene con:

1. **Serialización por *page table lock* (`ptl`)**: tanto el vaciado como la falta de escritura toman el mismo cerrojo de PTE; no hay ventana sin bloqueo.
2. **Decisión de COW por exclusividad real** (`PageAnonExclusive`, ≥ 5.19) en vez del bit *dirty*, de modo que `get_user_pages(FOLL_FORCE|FOLL_WRITE)` **no reutiliza** un mapeo COW hacia la página de solo lectura.
3. El *fix* original de 2016 introdujo `FOLL_COW` para distinguir un COW ya resuelto de uno pendiente.

`test_race.c` ejercita la versión **benigna** de esta carrera contra nuestro driver y comprueba que el descifrado por `.fault`, protegido por `store_lock`, nunca entrega una página a medio descifrar.

10. Aislamiento de memoria: `struct page`, tablas de páginas y bits de la MMU

- `struct page`: metadato por marco físico (`flags`, `_refcount`, `_mapcount`, `mapping`). Determina propiedad y nivel de compartición; un `_mapcount` incorrecto rompe el aislamiento COW.
- **Tablas** `PGD/P4D/PUD/PMD/PTE`: traducen la dirección virtual a física y **portan los bits de protección por página**, aislando el espacio de cada proceso (cada uno con su propio `PGD` / `CR3`).
- **Bits de la MMU:**
  - **R/W**: una página de solo lectura no puede escribirse (base de COW y de la protección de código).
  - **U/S** (User/Supervisor): impide que Ring 3 acceda a páginas de kernel; sustenta **SMEP/SMAP**.
  - **NX/XD** (No-Execute): impide **ejecutar** bytes en páginas de datos/pila. Es la defensa central contra **inyección de código**: aunque un atacante escriba *shellcode* en una región mapeada de datos, el bit NX provoca una falla al intentar ejecutarlo (política **W^X**).

Juntos garantizan que el contenido inyectado en memoria mapeada **no sea ejecutable**, que un proceso **no cruce** a memoria ajena o del kernel, y que las regiones de solo lectura **no se alteren**.

PARTE II — DISEÑO DE LA ESTRATEGIA CON APOYO DE INTELIGENCIA ARTIFICIAL

2.1 Flujo de trabajo asistido por IA

- **Exploración arquitectónica.** La IA se usó para mapear el terreno: qué *hooks* existen (`.mmap`, `vm_operations_struct.fault`, `page_mkwrite`, `security_mmap_file`), qué estructuras tocar (`vm_area_struct`, `struct page`), y para contrastar LKM vs. LSM vs. eBPF. **Rol: acelerador de lectura de cabeceras y del árbol de fuentes**, no fuente de verdad.
- **Generación y verificación de código C de bajo nivel.** Esqueletos de `file_operations`, el patrón `alloc_chrdev_region` / `cdev_add` / `class_create`, el manejador `.fault` y el uso de `kmap_local_page`. **Todo fragmento se compiló contra los *headers* reales** y se revisó firma por firma.
- **Casos de prueba.** La IA ayudó a diseñar los clientes (`test_access`, `test_readonly`, `test_race`) y el *benchmark*. Los **PoC se mantuvieron defensivos**: ejercitan la política del módulo (rechazos, concurrencia benigna), sin construir un exploit de escalada funcional.

2.2 Ingeniería de *prompts* y verificación crítica

Tabla de *prompts* estructurados (contexto · rol · restricción · resultado):

#	Contexto	Rol asignado	Restricción	Resultado esperado
P1	“Driver de caracteres en kernel 6.x que sirve páginas por demanda.”	Ingeniero de kernel Linux.	Solo APIs **exportadas** y vigentes en 6.x; citar la cabecera de cada símbolo.	Esqueleto de `vm_operations_struct.fault` + `.mmap` con firmas correctas.
P2	“Necesito impedir `mprotect(PROT_WRITE)` sobre un VMA de solo lectura.”	Revisor de seguridad de memoria.	Nada de modificar `vma->vm_flags` por asignación directa.	Uso de `vm_flags_clear(vma, VM_WRITE|VM_MAYWRITE)` y explicación.
P3	“Cifrado transparente de páginas en `.fault`.”	Arquitecto de subsistema de FS.	No exponer el claro a procesos no autorizados; señalar limitaciones.	Descifrado por página con control de `uid`; recomendación de `crypto_aead` real.
P4	“Medir el sobrecosto en page faults y syscalls.”	Ingeniero de rendimiento.	Usar `getrusage`, `perf`, `strace`, `ftrace`; nada de micro-benchmarks sesgados.	Metodología con `ru_minflt` y comparación contra mapeo anónimo.

2.3 Alucinaciones técnicas detectadas y corregidas

Durante la asistencia se detectaron y **rechazaron** sugerencias incorrectas o inseguras — evidencia de verificación crítica:

Sugerencia de la IA	Por qué es incorrecta / insegura	Corrección aplicada
`vma->vm_flags |= VM_WRITE;`	Desde 6.3 `vm_flags` es de solo lectura (`__private`); la asignación directa **no compila**.	`vm_flags_set/clear()`.
`class_create(THIS_MODULE, name)`	La forma de **dos argumentos** se eliminó en 6.4.	`class_create(name)` (un argumento).
Hook de `sys_call_table` deshabilitando WP: `write_cr0(read_cr0() & ~0x10000)`	Inseguro: rompe con *CR0 pinning*/CET, carreras, riesgo de *panic*.	Se descartó; se usa el `.mmap` del driver (y se recomienda LSM).
`.fault = f(struct vm_area_struct*, struct vm_fault*)`	Firma antigua; desde 4.11 es `f(struct vm_fault *vmf)`.	Firma de un argumento.
`kmap(page)` para copiar	`kmap()` está en desuso y es costoso.	`kmap_local_page()` / `kunmap_local()`.
`page_mkwrite(vma, vmf)`	Firma desactualizada.	`page_mkwrite(struct vm_fault *vmf)`.
“Exporta `handle_mm_fault` con `EXPORT_SYMBOL` y llámalo.”	Es símbolo del núcleo; no se re-exporta desde un módulo.	No se usa; se opera dentro del *callback* provisto.

2.4 Procedimiento de validación manual

1. **Compilación** contra `linux-headers-$(uname -r)` con `-Wall` → *clean build*.
2. **Verificación de firmas**: cada símbolo se cotejó en las cabeceras (`linux/mm.h`, `linux/fs.h`, `linux/highmem.h`).
3. **Guardas de versión** cuando aplica (`LINUX_VERSION_CODE`).
4. **Símbolos no resueltos**: `nm mmapguard.ko | grep ' U '` → todos son exports del kernel (`alloc_pages`, `crc32_le`, `from_kuid`, …).
5. **Prueba en VM desechable** y revisión de `dmesg` para descartar *warnings* o *panics* antes de cualquier uso en el equipo real.

2.5 Arquitectura seleccionada

Elección: módulo de kernel (LKM) con dispositivo de caracteres + `vm_operations_struct`. Es el punto de intercepción más limpio para controlar una región `mmap` concreta, sin parcheo de tablas y con capacidad de **transformar** el contenido (cifrado) —algo fuera del alcance de eBPF—.

```
ESPACIO DE USUARIO (Ring 3)
  proceso: fd = open("/dev/mmapguard"); p = mmap(fd)
       |  open/mmap/ioctl                          ^  lectura/escritura (MOV)
       v                                           |  (tras poblar la PTE)
------------------------------------------------- FRONTERA Ring3/Ring0 -------
ESPACIO DE KERNEL (Ring 0)   [ mmapguard.ko ]
  dev_mmap()  --> control de uid + politica RO (limpia VM_WRITE/VM_MAYWRITE)
       |            + set vm_ops   [la escritura se decide AQUI]
       v   (primer acceso => #PF => handle_mm_fault => do_fault)
  mg_vma_fault()  --> valida uid del que falla + DESCIFRA pagina => vmf->page
       |
       v   (munmap / cierre)
  mg_vma_close()  --> RECIFRA al almacen
  mg_audit (/proc/mmapguard)  <-- registro forense uid/pid/comm/tiempo
-------------------------------------------------------------------------------
  Almacen cifrado (respaldo)  +  CRC32 baseline (integridad tipo tripwire)
```

2.6 Matriz de compensaciones (*trade-offs*)

Enfoque	Latencia I/O / Page Fault	Profundidad de inspección	Cobertura	Estabilidad / Seguridad	Esfuerzo
Envoltorio en usuario (`LD_PRELOAD`)	Muy baja	Nula (eludible)	Solo libc, por proceso	Alta estab. / **nula seguridad**	Bajo
LKM parcheando `syscall_table`	Media	Media (sin contexto rico)	Global	**Baja** (panic/CET)	Alto
LSM (`file_mmap`)	Baja	Media-alta (allow/deny)	Global, apilable	Alta	Medio
BPF-LSM / kprobes	Baja	Alta para auditar/decidir	Global, dinámico	Alta	Medio
Driver + `vm_ops` (elegido)	Media (descifrado por página)	**Alta (transforma contenido)**	La región del dispositivo	Alta	Medio

Lectura: el enfoque elegido paga algo de latencia en el `.fault` (descifrado + auditoría) a cambio de la **única** capacidad que la política pura no da: **transformar** el contenido (cifrado) manteniendo control de acceso e integridad. Para política a nivel de sistema sobre archivos arbitrarios, se combinaría con un **BPF-LSM**.

PARTE III — IMPLEMENTACIÓN, RESULTADOS Y ARGUMENTACIÓN

3.1 Entregables de código

- **Módulo en C modularizado** (`kernel/`): `mg_core.c` (política + `vm_ops`), `mg_crypto.c` (cifrado/integridad), `mg_audit.c` (auditoría `/proc`), con cabeceras interna y UAPI. Compatible con **kernel 5.x / 6.x**.
- `Makefile` con *clean build* (`-Wall`), verificado contra `linux-headers-6.8.0-147-generic` **sin advertencias del código**.
- **Demostración** de intercepción, validación de permisos y cifrado/descifrado (`tests/` , `scripts/run_demo.sh`).
- **Scripts de prueba** que simulan acceso no autorizado (`uid` distinto), modificación indebida (detección por integridad) y carrera (`madvise(MADV_DONTNEED)` concurrente).

3.2 Justificación teórica: ¿por qué en la capa de kernel?

Una vez establecida la PTE, la lectura/escritura sobre la región mapeada se ejecuta **a velocidad de hardware, sin conmutar a Ring 0**. Por tanto:

- Una defensa en **espacio de usuario** (p. ej. envoltorio `LD_PRELOAD`) es **trivialmente eludible**: el atacante llama `mmap` / `syscall` directamente o precarga otra biblioteca.
- La defensa debe **arraigarse en Ring 0**, en el momento en que el kernel **crea el VMA** (`.mmap`) y **puebla las páginas** (`.fault` / `page_mkwrite`), que son los únicos puntos por los que todo acceso pasa obligatoriamente. Ahí residen el control de `uid` , el endurecimiento de `vm_flags` , el cifrado por demanda y la auditoría — ninguno realizable con garantías desde usuario.

3.3 Medición de impacto (metodología y salida de referencia)

Herramientas: `getrusage()` (`ru_minflt` = fallos de página menores), `clock_gettime(CLOCK_MONOTONIC)` (latencia), `strace -c` (conteo de syscalls) y `perf stat` (page-faults, ciclos). Script: `scripts/measure_overhead.sh`.

Metodología: `tests/bench.c` cronometra, por iteración, `mmap()` + tocar las 8 páginas (forzando los *page faults* que atiende el `.fault` con **descifrado + auditoría**) + `munmap()`, y lo compara contra el mismo patrón sobre memoria **anónima** (sin *hook*), aislando el sobrecosto neto de la defensa.

Resultados medidos (kernel `7.1.5+kali-amd64`, 2000 iteraciones × 8 páginas):

```
Benchmark MMAPGuard  (2000 iteraciones, 8 paginas/iter)
Ruta PROTEGIDA  :  81976.4 ns/iter | minflt total = 16002
Ruta ANONIMA    :   8881.2 ns/iter | minflt total = 16000
Sobrecosto abs. :  73095.2 ns/iter
Sobrecosto rel. :    823.0 %
```

El conteo de fallos menores (`ru_minflt`) es prácticamente idéntico (16002 vs 16000): **la defensa no genera fallos de página adicionales** — exactamente los 8 esperados por iteración (16000 = 2000 × 8). El sobrecosto no está en *más* fallos, sino en el *trabajo por fallo*.

Análisis del sobrecosto (≈73 µs/iter). Se descompone en tres fuentes, todas en la ruta del `#PF`, no en syscalls (`strace -c` confirma igual número de `mmap` / `munmap` en ambas rutas):

1. **Descifrado por demanda**: 8 páginas × 4 KiB de keystream XOR por iteración.
2. **Auditoría**: registro de cada `FAULT` / `MMAP_OK` / `SYNC` en el ring bajo *spinlock*, más el `pr_info` por evento al log del kernel (el `printk` es, con diferencia, el componente más caro; desactivarlo deja el costo dominado solo por el descifrado).
3. **Recifrado al cierre**: en cada `munmap` se recifran las 8 páginas vivas y se actualiza el almacén (los eventos `SYNC` del registro).

Naturaleza amortizable. El benchmark es un *peor caso* deliberado: mapea, toca una vez cada página y desmapea, de modo que **cada acceso es un primer acceso** (todo es fallo + descifrado + cierre). En uso real, una región se mapea una vez y se accede miles de veces: solo el primer toque de cada página paga el descifrado; los accesos siguientes, ya con la PTE poblada, corren **a velocidad de hardware, sin sobrecosto**. Por eso el 823 % medido aquí es el techo, y el costo relativo **tiende a cero** a medida que crece la relación accesos/fallos. El contador `FAULT` de `/proc/mmapguard` (cientos de miles tras la prueba de concurrencia, que expulsa la página en bucle con `MADV_DONTNEED`) ilustra justo ese punto: forzar refaults constantes es lo que encarece la ruta.

3.4 Limitaciones y trabajo futuro

Limitaciones actuales (declaradas): - Cifrado **XOR didáctico**, no productivo (ver más abajo). - Recifrado en `.close` del VMA, no en cada `msync` con *backing* de `address_space`. - Cubre la región del dispositivo, no archivos arbitrarios del sistema.

Vectores aún descubiertos: - Un proceso **autorizado y malicioso** puede volcar el claro mientras tiene la página mapeada (problema intrínseco: si puede leer, puede copiar). - Acceso físico / DMA malicioso que lea RAM fuera del control de la MMU. - Ataques de **canal lateral** (p. ej. temporización de *page faults*).

Trabajo futuro (mitigaciones por capa): - **Software/kernel:** sustituir el XOR por **AES-GCM** vía `crypto_aead` de la Crypto API; mover la política a un **BPF-LSM** para cobertura de sistema; claves por archivo ligadas al *keyring* (modelo fscrypt). - **Hardware/CPU:** páginas de memoria con **Intel MPK/PKU** (`pkey_mprotect`) para dominios de protección por hilo; **CET** contra reutilización de código; **NX/W^X** estricto. - **Hipervisor/enclaves:** cifrado de memoria **AMD SEV-SNP** / **Intel TDX** para proteger la RAM frente al propio anfitrión; **Intel SGX** para enclaves de descifrado; **IOMMU** para contener DMA malicioso.

3.5 Argumentación final

`MMAPGuard` demuestra, con código que **compila limpio** contra kernel 6.x, los cuatro puntos de intervención que el reto exige: **control de acceso** (`.mmap` + `.fault` por `uid`), **prevención de modificaciones no autorizadas** (política RO con endurecimiento de `vm_flags` e integridad *tripwire*), **cifrado transparente** por demanda y **auditoría** forense en `/proc/mmapguard`. La elección de la capa de kernel no es estética: es la **única** capa donde la defensa es **inevitable** para el proceso vigilado, porque intercepta la creación del VMA y el poblado de páginas — los dos cuellos por los que, obligatoriamente, pasa todo acceso a una región `mmap`.

Anexo — Rúbrica y cómo este entregable la cubre

Criterio	Evidencia en este entregable
10 preguntas (30%)	Parte I: respuestas con flujo exacto de eventos y estructuras (`vm_area_struct`, MMU, PTEs, `handle_mm_fault`, `PageAnonExclusive`).
Estrategia y uso de IA (30%)	Parte II: flujo con IA, tabla de 4 *prompts*, 7 alucinaciones detectadas y corregidas, validación, arquitectura y matriz de *trade-offs*.
Solución técnica en kernel (40%)	`kernel/` (compila limpio), `tests/` + `scripts/` (demostración y medición), justificación, metodología de *overhead* y limitaciones/trabajo futuro.

Entrega: repositorio github.com/Juanp-la/mmapguard-so2026b con `kernel/`, `tests/`, `scripts/`, `Makefile`s, `README.md` y este `INFORME.md` (exportable a PDF). El `insmod` y las mediciones finales se ejecutan en la VM/ equipo con *headers* del kernel (ver README §4–§5).